Nexus / Artifact Repository — 3–4 YOE Scenario-Based Q&A

1. Artifact upload to Nexus is failing. What will you check?
Ans: First, I check the build output and identify why the artifact upload is failing. Then I verify the Nexus repository URL, repository name, artifact version, and authentication details.
I also check whether the user has permission to upload artifacts and whether the Nexus server is reachable from the Jenkins agent. After fixing the issue, I run the build again and verify that the artifact is uploaded successfully.
Simple ga gurthupettuko: Artifact upload fail ayithe → first build output lo error chudu → Nexus URL and repository verify cheyyi → artifact version check cheyyi → authentication and upload permission chudu → Jenkins agent nunchi Nexus reach avutunda check cheyyi → issue fix chesi artifact malli upload cheyyi.
Eng: If artifact upload fails → first check the build error → verify the Nexus URL and repository → check the artifact version → verify authentication and upload permission → check Nexus connectivity from Jenkins → fix the issue and upload the artifact again.
Example:
mvn deploy → artifact upload attempt cheyyi.
Build output → Nexus upload error ekkada vachindo chudu.
curl http://nexus:8081 → Jenkins agent nunchi Nexus reach avutunda check cheyyi.
Nexus → Repository → upload permission verify cheyyi.
Issue fix chesi → mvn deploy → artifact upload ayyinda verify cheyyi.

---

2. Artifact download from Nexus is failing. What will you check?
Ans: First, I check the build output and identify which dependency or artifact cannot be downloaded. Then I verify the repository URL, artifact name, group ID, artifact ID, and version.
I also check Nexus authentication, repository availability, and network connectivity. If the requested artifact does not exist in the repository, I verify whether the correct version is being requested.
Simple ga gurthupettuko: Artifact download fail ayithe → first build error chudu → ye artifact download avvatledo identify cheyyi → repository URL and artifact details verify cheyyi → authentication check cheyyi → Nexus available unda chudu → requested version repository lo unda verify cheyyi → issue fix chesi build malli run cheyyi.
Eng: If artifact download fails → first check the build error → identify the missing artifact → verify the repository URL and artifact details → check authentication → verify Nexus availability → confirm the requested version exists → fix the issue and rerun the build.
Example:
mvn clean install → dependency download attempt cheyyi.
Build output → which artifact download failed chudu.
curl http://nexus:8081 → Nexus reachable aa check cheyyi.
Nexus Repository → artifact and version exist avutunnaya verify cheyyi.
Issue fix chesi → mvn clean install → dependency download ayyinda verify cheyyi.

---

3. Nexus authentication is failing. What will you check?
Ans: First, I check whether the username and password or token being used are correct. Then I verify whether the credentials configured in Jenkins or Maven are pointing to the correct Nexus server.
I also check whether the user is active and has the required repository permissions. If the credentials are invalid or expired, I update them securely and test the connection again.
Simple ga gurthupettuko: Nexus authentication fail ayithe → first credentials correct aa chudu → correct Nexus server ki point avutunnaya check cheyyi → user active unda verify cheyyi → repository permissions chudu → credentials expired or invalid aa check cheyyi → correct credentials update chesi test cheyyi.
Eng: If Nexus authentication fails → first verify the credentials → confirm the correct Nexus server → check whether the user is active → verify repository permissions → check whether credentials are expired or invalid → update valid credentials and test again.
Example:
Jenkins → Manage Credentials → Nexus credentials check cheyyi.
Maven settings.xml → Nexus server configuration verify cheyyi.
Nexus → Security → Users → user active aa chudu.
Nexus → Repository permissions → required access unda verify cheyyi.
Correct credentials configure chesi → mvn deploy → authentication successful aa check cheyyi.

---

4. Wrong artifact version is being downloaded. What will you check?
Ans: First, I check the dependency version configured in the application build file. Then I verify whether the requested version exists in Nexus and whether another dependency is bringing a different version.
I also check the Maven dependency tree to identify which dependency is actually being used. After identifying the wrong version, I correct the dependency configuration and rebuild the application.
Simple ga gurthupettuko: Wrong artifact version vaste → first build file lo version chudu → Nexus lo aa version unda verify cheyyi → dependency vere version tiskosthunda check cheyyi → dependency tree chudu → correct version configure chesi build malli run cheyyi.
Eng: If the wrong artifact version is downloaded → first check the version in the build file → verify the version exists in Nexus → check whether another dependency brings a different version → review the dependency tree → configure the correct version and rebuild.
Example:
mvn dependency:tree → actual dependency version chudu.
pom.xml → required version verify cheyyi.
Nexus → Repository → artifact versions chudu.
Correct version configure chesi → mvn clean install → dependency version verify cheyyi.

5. Nexus repository is unavailable. What will you check?
Ans: First, I check whether the Nexus service is running and whether the server is reachable. Then I check the Nexus logs for application, database, or storage-related errors.
I also verify CPU, memory, and disk space because insufficient resources can affect Nexus. If Nexus uses an external database or storage, I check those dependencies as well.
Simple ga gurthupettuko: Nexus unavailable ayithe → first service running unda chudu → server reach avutunda check cheyyi → Nexus logs chudu → CPU, memory and disk verify cheyyi → database or storage dependency check cheyyi → issue fix chesi Nexus access verify cheyyi.
Eng: If Nexus is unavailable → first check whether the service is running → verify server connectivity → check Nexus logs → verify CPU, memory, and disk → check database or storage dependencies → fix the issue and verify Nexus access.
Example:
systemctl status nexus → Nexus service status chudu.
curl http://localhost:8081 → Nexus locally respond avutunda check cheyyi.
df -h → disk space verify cheyyi.
free -m → memory usage check cheyyi.
Logs chusi → actual error identify chesi → issue fix cheyyi.

---

6. Jenkins cannot upload artifacts to Nexus. What will you check?
Ans: First, I check the Jenkins console output and identify whether the issue is authentication, repository configuration, network connectivity, or permissions. Then I verify the Nexus credentials configured in Jenkins.
I also check the repository URL and confirm that the Jenkins agent can reach Nexus. Finally, I verify that the Nexus user has permission to upload artifacts.
Simple ga gurthupettuko: Jenkins nunchi Nexus upload avvakapothe → first console output chudu → authentication or repository issue identify cheyyi → Jenkins credentials verify cheyyi → repository URL check cheyyi → Jenkins agent connectivity chudu → upload permission verify cheyyi → issue fix chesi pipeline malli run cheyyi.
Eng: If Jenkins cannot upload artifacts to Nexus → first check the console output → identify authentication or repository issues → verify Jenkins credentials → check the repository URL → verify Jenkins connectivity → check upload permission → fix the issue and rerun the pipeline.
Example:
Jenkins → Console Output → Nexus upload error chudu.
Jenkins → Manage Credentials → Nexus credentials verify cheyyi.
curl http://nexus:8081 → connectivity check cheyyi.
Nexus → Repository permissions → Jenkins user upload permission chudu.
mvn deploy → upload successful aa verify cheyyi.

---

7. Maven is unable to download dependencies from Nexus. What will you check?
Ans: First, I check the Maven error and identify which dependency is failing to download. Then I verify the repository configuration in settings.xml or pom.xml.
I also check whether the dependency exists in Nexus and whether Maven has the required authentication. If Nexus is reachable and the dependency exists, I check whether the configured version or repository URL is correct.
Simple ga gurthupettuko: Maven dependency download fail ayithe → first Maven error chudu → failed dependency identify cheyyi → settings.xml or pom.xml repository configuration check cheyyi → Nexus lo dependency unda verify cheyyi → authentication chudu → version and repository URL correct aa check cheyyi → issue fix chesi Maven malli run cheyyi.
Eng: If Maven cannot download a dependency → first check the Maven error → identify the failed dependency → verify repository configuration → confirm the dependency exists in Nexus → check authentication → verify the version and repository URL → fix the issue and rerun Maven.
Example:
mvn clean install → dependency download error chudu.
cat ~/.m2/settings.xml → Nexus repository configuration check cheyyi.
Nexus → Repository → dependency version verify cheyyi.
mvn dependency:tree → dependency version check cheyyi.
Issue fix chesi → mvn clean install → dependency download verify cheyyi.

---

8. Artifact is uploaded successfully, but users cannot download it. What will you check?
Ans: First, I verify that the artifact is present in the correct Nexus repository. Then I check the repository permissions and confirm that the users have read access.
I also verify the artifact coordinates and repository URL being used by the users. If the repository is available but access is denied, I correct the required permissions.
Simple ga gurthupettuko: Artifact upload ayina download avvakapothe → first artifact correct repository lo unda chudu → repository read permission check cheyyi → artifact coordinates verify cheyyi → users correct repository URL use chesthunnara chudu → access denied ayithe permissions correct cheyyi.
Eng: If the artifact is uploaded but cannot be downloaded → first verify the artifact exists in the correct repository → check read permissions → verify artifact coordinates → confirm users are using the correct repository URL → fix permissions if access is denied.
Example:
Nexus → Repository → artifact search cheyyi.
Artifact → group ID, artifact ID and version verify cheyyi.
Nexus Security → user permissions check cheyyi.
User machine → repository URL correct aa verify cheyyi.
Permission fix chesi → artifact download malli test cheyyi.

9. Nexus disk space is getting full. What will you check?
Ans: First, I check the disk usage and identify which repositories or files are consuming the most space. Then I review old artifacts, unused components, and repository retention policies.
I also check whether cleanup policies are configured correctly. Before deleting anything, I verify that the artifacts are not required by active applications or release processes.
Simple ga gurthupettuko: Nexus disk full avutunte → first disk usage chudu → ye repository ekkuva space use chesthundo identify cheyyi → old and unused artifacts check cheyyi → cleanup policy verify cheyyi → required artifacts delete kakunda cleanup cheyyi → disk space malli verify cheyyi.
Eng: If Nexus disk space is getting full → first check disk usage → identify the repository consuming more space → review old and unused artifacts → verify cleanup policies → clean up without deleting required artifacts → verify disk space again.
Example:
df -h → Nexus disk usage chudu.
Nexus → Repository → storage usage identify cheyyi.
Nexus → Cleanup Policies → configured policy check cheyyi.
Old unused artifacts → safe to remove aa verify cheyyi.
Cleanup tarvatha → df -h → free space verify cheyyi.

---

10. Nexus Docker image push is failing. What will you check?
Ans: First, I check whether the Docker image is tagged with the correct Nexus registry address and repository name. Then I verify Docker authentication with the Nexus registry.
I also check whether the user has permission to push images and whether the Docker repository is available. Finally, I verify network connectivity and retry the push after fixing the issue.
Simple ga gurthupettuko: Nexus Docker image push fail ayithe → first image tag lo Nexus registry correct aa chudu → Docker login verify cheyyi → Docker repository check cheyyi → push permission verify cheyyi → network connectivity chudu → issue fix chesi image push malli run cheyyi.
Eng: If Docker image push to Nexus fails → first verify the Nexus registry in the image tag → check Docker login → verify the Docker repository → check push permissions → verify network connectivity → fix the issue and push the image again.
Example:
docker login nexus:8082 → Nexus Docker registry login check cheyyi.
docker tag myapp:latest nexus:8082/myapp:latest → image tag verify cheyyi.
docker push nexus:8082/myapp:latest → image push test cheyyi.
Nexus → Docker Repository → image available aa verify cheyyi.

---

11. Nexus Docker image pull is failing. What will you check?
Ans: First, I verify the image name, registry address, repository, and tag. Then I check whether the image exists in the Nexus Docker repository.
I also verify Docker authentication, read permissions, and network connectivity. If the repository is private, I make sure the system pulling the image has the required access.
Simple ga gurthupettuko: Nexus Docker image pull fail ayithe → first registry, repository and tag verify cheyyi → Nexus lo image unda chudu → Docker login check cheyyi → read permission verify cheyyi → network connectivity check cheyyi → private repository ayithe access confirm cheyyi → image malli pull cheyyi.
Eng: If Docker image pull from Nexus fails → first verify the registry, repository, and tag → confirm the image exists → check Docker login → verify read permission → check network connectivity → confirm private repository access → pull the image again.
Example:
docker login nexus:8082 → registry authentication check cheyyi.
docker pull nexus:8082/myapp:latest → image pull test cheyyi.
Nexus → Docker Repository → image and tag verify cheyyi.
Permission check chesi → docker pull malli run cheyyi.

---

12. Nexus is showing the wrong artifact version. What will you check?
Ans: First, I verify the artifact metadata and the version used during the build. Then I check whether Jenkins is publishing the expected version or overwriting an existing version.
I also review the Maven configuration and build pipeline variables that control the artifact version. After correcting the version configuration, I publish the artifact again and verify the repository.
Simple ga gurthupettuko: Wrong version Nexus lo unte → first artifact version chudu → Jenkins ye version publish chesthundo check cheyyi → Maven configuration verify cheyyi → pipeline variables check cheyyi → existing version overwrite avutunda chudu → correct version publish chesi Nexus lo verify cheyyi.
Eng: If Nexus shows the wrong artifact version → first check the artifact version → verify which version Jenkins publishes → check Maven configuration → review pipeline variables → check whether an existing version is being overwritten → publish the correct version and verify it in Nexus.
Example:
mvn help:evaluate -Dexpression=project.version -q -DforceStdout → build version chudu.
pom.xml → project version verify cheyyi.
Jenkins → Build parameters → version value check cheyyi.
Nexus → artifact → uploaded version verify cheyyi.

---

13. Nexus cleanup policy is not removing old artifacts. What will you check?
Ans: First, I check whether the cleanup policy is created and assigned to the correct repository. Then I verify the policy conditions such as artifact age or number of versions.
I also check whether the cleanup task is running successfully and review Nexus logs if required. I make sure important release artifacts are protected before changing the policy.

Simple ga gurthupettuko: Cleanup policy work avvakapothe → first policy create ayyinda chudu → correct repository ki assign ayyinda check cheyyi → age and version conditions verify cheyyi → cleanup task run avutunda chudu → logs check cheyyi → important artifacts protect chesi policy correct cheyyi.
Eng: If the cleanup policy is not removing old artifacts → first verify the policy exists → confirm it is assigned to the correct repository → check age and version conditions → verify the cleanup task runs → review logs → protect important artifacts and correct the policy.
Example:
Nexus → Repository → Cleanup Policy → assigned policy chudu.
Cleanup Policy → conditions verify cheyyi.
Nexus → Tasks → cleanup task status check cheyyi.
Task fail ayithe → Nexus logs chusi error identify cheyyi.
Policy correct chesi → cleanup task malli run cheyyi.

---

14. Nexus repository is returning 403 Forbidden. What will you check?
Ans: First, I check whether the user is authenticated correctly. Then I verify the permissions assigned to that user for the specific repository and action.
I also check whether the user has read or write access as required. If authentication is successful but access is denied, I correct the repository permissions.
Simple ga gurthupettuko: Nexus 403 Forbidden vaste → first authentication check cheyyi → user permissions chudu → correct repository access unda verify cheyyi → read or write permission required aa check cheyyi → permission issue ayithe correct chesi request malli test cheyyi.
Eng: If Nexus returns 403 Forbidden → first check authentication → verify user permissions → confirm access to the correct repository → check required read or write permission → correct the permission issue and test again.
Example:
Nexus → Security → Users → user identify cheyyi.
Nexus → Roles/Privileges → repository permission check cheyyi.
403 request → read or write action identify cheyyi.
Required permission add chesi → request malli test cheyyi.

---

15. Nexus repository is returning 404 Not Found. What will you check?
Ans: First, I verify the repository URL and the artifact coordinates being requested. Then I check whether the artifact actually exists in Nexus with the same group ID, artifact ID, and version.
I also check whether the request is going to the correct repository. If the artifact does not exist, I verify the publishing process.
Simple ga gurthupettuko: Nexus 404 vaste → first repository URL check cheyyi → artifact coordinates verify cheyyi → group ID, artifact ID and version chudu → correct repository ki request velthunda check cheyyi → artifact Nexus lo unda verify cheyyi → lekapothe publishing issue check cheyyi.
Eng: If Nexus returns 404 Not Found → first check the repository URL → verify artifact coordinates → check group ID, artifact ID, and version → confirm the correct repository is being used → verify the artifact exists → check the publishing process if it is missing.
Example:
Nexus → Repository → artifact search cheyyi.
Artifact → group ID, artifact ID and version compare cheyyi.
Maven URL → correct repository endpoint use chesthunnara chudu.
Artifact missing ayithe → Jenkins mvn deploy output check cheyyi.

---

16. Jenkins build is downloading artifacts from the wrong Nexus repository. What will you check?
Ans: First, I check the Maven repository configuration in settings.xml and pom.xml. Then I verify the repository URL and mirror configuration being used by Jenkins.
I also check whether Jenkins is using a different Maven settings file than expected. After correcting the repository configuration, I run the build again and verify the download source.
Simple ga gurthupettuko: Wrong Nexus repository nunchi download ayithe → first settings.xml and pom.xml chudu → repository URL verify cheyyi → mirror configuration check cheyyi → Jenkins ye settings file use chesthundo verify cheyyi → correct repository configure chesi build malli run cheyyi.
Eng: If Jenkins downloads from the wrong Nexus repository → first check settings.xml and pom.xml → verify the repository URL → check mirror configuration → confirm which settings file Jenkins uses → configure the correct repository and rerun the build.
Example:
cat ~/.m2/settings.xml → repository and mirror configuration chudu.
pom.xml → repository URL check cheyyi.
Jenkins → Maven configuration → settings file verify cheyyi.
mvn clean install → correct Nexus repository nunchi download avutunda check cheyyi.

---

17. Nexus artifact upload is very slow. What will you check?
Ans: First, I check the artifact size and network connectivity between Jenkins and Nexus. Then I verify Nexus server CPU, memory, disk performance, and current load.
I also check whether the repository or storage backend is performing slowly. If the network or server is the bottleneck, I address that issue and test the upload again.
Simple ga gurthupettuko: Artifact upload slow ga unte → first artifact size chudu → Jenkins to Nexus network check cheyyi → Nexus CPU and memory verify cheyyi → disk performance chudu → repository or storage load check cheyyi → bottleneck fix chesi upload malli test cheyyi.
Eng: If artifact upload is slow → first check artifact size → verify network between Jenkins and Nexus → check Nexus CPU and memory → review disk performance → check repository or storage load → fix the bottleneck and test the upload again.
Example:
ls -lh target/ → artifact size chudu.

curl -I http://nexus:8081 → basic connectivity check cheyyi.
top → Nexus server CPU and memory usage chudu.
df -h → disk space verify cheyyi.
Issue identify chesi → mvn deploy → upload speed malli verify cheyyi.

---

18. Nexus artifact is corrupted or cannot be used after download. What will you check?
Ans: First, I verify whether the downloaded artifact is complete and compare its checksum with the repository artifact when available. Then I check whether the artifact was uploaded correctly.
I also verify network interruptions during upload or download and check whether the local Maven cache contains a corrupted copy. If required, I clear the affected local cache and download the artifact again.
Simple ga gurthupettuko: Artifact corrupt ga unte → first downloaded file verify cheyyi → checksum compare cheyyi → Nexus lo original artifact check cheyyi → upload or download interruption unda chudu → local Maven cache corrupt aa verify cheyyi → cache clear chesi artifact malli download cheyyi.
Eng: If the artifact is corrupted → first verify the downloaded file → compare the checksum → check the original artifact in Nexus → verify upload or download interruptions → check the local Maven cache → clear the affected cache and download the artifact again.
Example:
sha256sum myapp.jar → downloaded artifact checksum chudu.
Nexus → artifact → original file verify cheyyi.
rm -rf ~/.m2/repository/group/artifact/version → affected local cache remove cheyyi.
mvn clean install → artifact malli download cheyyi.
Application build successful aa verify cheyyi.

---

19. Nexus is running out of storage because of Docker images. What will you do?
Ans: First, I identify which Docker repositories and image tags are consuming the most storage. Then I check old, unused image versions and verify whether cleanup policies are configured.
I make sure images currently used by production applications are not removed. After confirming the safe cleanup candidates, I apply the cleanup policy and verify the storage usage again.
Simple ga gurthupettuko: Docker images valla Nexus storage full ayithe → first ye Docker repository ekkuva space use chesthundo chudu → old image tags identify cheyyi → production lo use chesthunna images protect cheyyi → cleanup policy check cheyyi → safe images cleanup cheyyi → storage malli verify cheyyi.
Eng: If Nexus storage is filling because of Docker images → first identify the Docker repository using more space → review old image tags → protect production images → check the cleanup policy → remove safe unused images → verify storage again.
Example:
Nexus → Docker Repository → image tags and storage usage chudu.
Production deployment → currently used image tags identify cheyyi.
Nexus → Cleanup Policy → old image cleanup configuration check cheyyi.
Safe unused images cleanup chesi → df -h → storage verify cheyyi.

---

20. How will you troubleshoot a Nexus issue in a production CI/CD pipeline?
Ans: First, I check the Jenkins console output and identify whether the issue is related to artifact upload, download, authentication, permissions, repository availability, or network connectivity.
Then I verify Nexus server health, repository configuration, credentials, artifact versions, and storage. After identifying the root cause, I fix the issue, rerun the pipeline, and verify that the artifact is correctly uploaded or downloaded.
Simple ga gurthupettuko: Production pipeline lo Nexus issue vaste → first Jenkins logs chudu → upload or download issue identify cheyyi → authentication and permissions check cheyyi → repository and artifact version verify cheyyi → Nexus health and storage chudu → network check cheyyi → root cause fix chesi pipeline malli run cheyyi → artifact correct ga upload or download ayyinda verify cheyyi.
Eng: If there is a Nexus issue in a production CI/CD pipeline → first check Jenkins logs → identify upload or download issues → verify authentication and permissions → check repository and artifact version → verify Nexus health and storage → check network connectivity → fix the root cause and rerun the pipeline → confirm the artifact is uploaded or downloaded correctly.
Example:
Jenkins → Failed Build → Console Output → Nexus error identify cheyyi.
curl http://nexus:8081 → Nexus reachable aa check cheyyi.
Jenkins Credentials → authentication verify cheyyi.
Nexus → Repository → artifact and version check cheyyi.
df -h → storage issue unda chudu.
Issue fix chesi → Jenkins Build Now → pipeline malli run cheyyi.
Nexus → artifact → upload/download successful aa final ga verify cheyyi.
